

Current Project Architecture & Refactoring Proposal

This document outlines the current state of the application architecture, analyzes the function distribution across files, and highlights architectural flaws. It then proposes a cleaner, modular architecture to solve the current scaling and maintenance issues.

1. Current Architecture Overview

The system is a Python-based algorithmic trading and options scanning platform integrated with the "Dhan" API (via `Dhan\_Tradehull.py` and `Dhan\_websocket.py`). It orchestrates market scanning, technical indicator processing, risk evaluation, and trade calculation.

Currently, the logic is highly coupled. Files like `Market\_Scanner.py` and `MainEngine.py` are acting as "God objects," aggregating disparate responsibilities such as fetching chains, computing indicators, formatting console output, sending excel updates, and triggering strategies.

1.1 System Modules & File Responsibilities

Below is the breakdown of all files and the functions they encapsulate:

```
#### **Core Data & Broker Connectors (Off Limits / Isolated)**
* **`Dhan_Tradehull.py`**
  * `class Tradehull`: (Given by broker, off-limits for now)
* **`Dhan_websocket.py`** (WebSocket Client & Excel Bridge)
  * `setup_websocket_logger()`
  * `get_client_credentials()`
  * `initialize_websocket_excel()`
  * `read_symbols_from_sheet()`
  * `run_websocket()`

#### **Market Scanning & Execution (The "God Classes")**
* **`Market_Scanner.py`** (818 lines) - Mixing state, UI/Print, Excel logging, and data fetching.
  * *State/Utility:* `set_shared_state()`, `get_shared_state()`, `maybe_release_state_mute()`
  * *Excel/Syncing:* `_log_excel_sync()`, `_websocket_workbook()`, `_sheet_last_data_row()`, `_normalize_sheet_exchange()`, `_sync_excel_ltp()`
  * *UI/Logging:* `print_watchlist_scan()`, `log_rsi_trend_mismatch()`, `log_rsi_band_alerts()`, `print_heartbeat()`
  * *Data Fetching:* `get_nifty_lm_cached()`, `calculate_live_pcr()`, `_fetch_index_chain()`, `fetch_chain()`, `fetch_sensex_chain()`, `chains_for_excel()`, `await_915_market_open()`
* **`MainEngine.py`** (610 lines) - Mixes configuration, credentials, order execution, and monitoring loops.
  * `load_cred_env()`, `get_live_client()`
  * `release_scanner_lock()`, `pause_engine_until_continue()`
  * `print_signal()`
  * `_fetch_entry_oi()`, `execute_verified_signal()`
  * `monitor_signals()`, `run_main_engine()`
* **`scanner_excel.py`** (1144 lines) - Massive file doing parsing, alerting, and Excel manipulation.
```

```

    * *Data Parsing:* `_num()`, `_quote_row_from_dhan()`,
`merge_quote_rows()`, `_quote_row_from_chain_option()`,
`build_quote_map_from_chain()`, `_build_oi_map_from_chain()`,
`build_chain_maps()`, `_build_combined_chain()`
    * *Futuers & Options Utils:* `_normalize_exchange()`,
`_parse_fno_symbol()`, `_expiry_to_tokens()`, `_fetch_active_expiries()`,
`is_fno_symbol()`, `_underlying_step()`, `_fno_exchange_for_symbol()`
    * *Strike Windows:* `_select_options_near_atm()`,
`filter_chain_strike_window()`, `_strike_window_symbols()`,
`refresh_strike_window_on_websocket()`, `_refresh_itm_rows_on_websocket()`
    * *Alerts & Logging:* `class HourlySignalLogger`, `class
OiVolumeAlertLogger`, `_oi_vol_today_csv()`, `_oi_vol_csv_paths()`,
`format_signed_int()`, `_format_oi_poll_message()`,
`format_vol_snapshot_message()`, `_run_oi_volume_monitor()`,
`play_alert_sound()`, `_print_alert_sound_help()`

```

Strategy & Risk Algorithms

```

* **`Option_strategy_core.py`** (511 lines) - Heavy strategy
implementation.
    * `_get_time_slot_and_multiplier()`, `_validate_itm_distance()`,
`validate_oi_barrier_distance()`, `_validate_volume_support()`,
`check_volume_ema_cross()`, `_check_oi_change_alert()`
    * `_build_score()`, `_build_and_score_contract()`,
`calculate_rsi_series()`, `_calculate_rsi()`, `_detect_trigger_lm()`,
`explain_trigger_failure()`, `_verify_trade_calculations()`
* **`Chain_Analyzer.py`**
    * `_vol_sign_label()`, `_process_and_analyze_chain()`,
`get_latest_chain_findings()`, `_is_safe_to_trade()`
* **`Indicators.py`** (Duplicates RSI logic from Option_strategy_core)
    * `_calculate_rsi_series()`, `_calculate_rsi()`, `_add_vwap()`,
`add_ema()`, `_calculate_volume_ema()`, `_parse_discrete_lm_volume()`
* **`Chop_Mode.py`**
    * `_calculate_adx()`, `_calculate_bollinger_bands()`,
`check_for_chop_reversion()`, `_commit_theta_dodge_signal()`
* **`Risk_Engine.py`**
    * `_detect_gap_risk()`, `_gap_cooldown_active()`, `_vwap_overextended()`,
`candle_exhausted()`, `_recent_rsi_exhaustion()`, `_allow_trade()`,
`explain_allow_trade_block()`
* **`RiseFall_sub.py`**
    * `_evaluate_ce_trigger()`, `_explain_ce_failure()`,
`evaluate_pe_trigger()`, `_explain_pe_failure()`

```

Calculations & Monitoring

```

* **`Price_Check.py`**
    * `_get_live_client()`, `_parse_arguments()`, `_calculate_oi_metric()`
* **`Trade_Calculator.py`**
    * `_get_lot_size()`, `_calculate_trade_parameters()`
* **`Monitor_Engine.py`**
    * `_execute_monitoring()`

```

Utilities & Comm

```

* **`excel_ledger_orderbook.py`**
    * `_record_trade()`
* **`Communication.py`**
    * `_telegram_log()`, `class DiscordNotifier`
* **`SafetyLogger.py`**
    * `_setup_logger()`, `_log_error()`, `_log_error_with_context()`,
`log_warning()`, `_log_info()`

```

```
* **`System_Config.py`**
* (No functions, likely stores configuration constants)

#### **Testing / Simulation**
* **`NSE_Option_Scanner_Backtest.py`**
* `generate_mock_lm_data()`, `generate_mock_option_chain()`,
`get_historical_data_from_dhan()`, `backtest_strategy()`, `main()`
* **`Backtest.py`**
```

2. Current Architectural Flaws

As the codebase has grown, several critical anti-patterns have emerged:

2.1 "God Classes" & Lack of Separation of Concerns

```
* `scanner_excel.py` (1,144 lines) and `Market_Scanner.py` (818 lines)
are handling far too many responsibilities. `scanner_excel.py` mixes UI
audio alerts (`play_alert_sound`), WebSocket chain filtering
(`refresh_strike_window_on_websocket`), and underlying string parsing
(`parse_fno_symbol`).
* `Market_Scanner.py` is doing UI Output (`print_heartbeat`), Excel
synchronization (`sync_excel_ltp`), and Data fetching (`fetch_chain`).
```

2.2 Duplication of Logic (DRY Violations)

```
* `calculate_rsi()` and `calculate_rsi_series()` exist in both
`Indicators.py` and `Option_strategy_core.py`.
* Multiple files are loading credentials and initializing clients
independently instead of relying on a centralized dependency injector:
* `MainEngine.py` -> `get_live_client()`, `load_cred_env()`
* `Price_Check.py` -> `get_live_client()`
* `Dhan_websocket.py` -> `get_client_credentials()`
```

2.3 Tight Coupling with Excel Output

```
* Options chain logic, algorithmic strategy engines, and the WebSocket
stream are inherently tied to Excel spreadsheet formatting and syncing.
This makes running the system head-less (without Excel) or testing it
incredibly difficult.
```

2.4 State Management Chaos

```
* `Market_Scanner.py` uses global-like getters/setters
(`set_shared_state`, `get_shared_state`) which causes race conditions in
asynchronous loops and makes testability near impossible.
```

3. Proposed Architecture (Refactoring Plan)

To reduce the size of these files and make the system robust, we need to compartmentalize the code based on the **Single Responsibility Principle**.

We will modularize the project into distinct domains:

Domain 1: `Core Data Layer` (API & State)

```
**Goal:** Handle all credential injection, global state, and Broker APIs
(Dhan API).
```

```
*   **`session_manager.py` (NEW):** Replaces scattered `load_cred_env`
and `get_live_client()`. It bootstraps the `Tradehull` client once and
makes it available system-wide.
*   **`dhan_api_client.py` (NEW):** Wraps raw `Tradehull` calls.
`fetch_chain()`, `get_nifty_lm_cached()`, etc., move here from
`Market_Scanner.py`.
*   **`StateManager.py` (NEW):** Takes over `get_shared_state` /
`set_shared_state` into a thread-safe singleton or state dictionary.
```

Domain 2: `Technical Indicators Layer`

```
**Goal:** A purely mathematical, side-effect-free library.
*   **`Indicators.py`:** Should strictly own all `RSI`, `VWAP`, `EMA`,
`Bollinger Bands`, and `ADX`.
*   *Action:* Remove duplicate RSI functions from
`Option_strategy_core.py`. Move `calculate_adx` and
`calculate_bollinger_bands` from `Chop_Mode.py` to `Indicators.py`.
```

Domain 3: `Strategy & Validation Engine Layer`

```
**Goal:** Takes market state & indicators and outputs Trade Signals. Does
NOT print to console or write to Excel.
*   **`Strategy_Core.py` (Refactored `Option_strategy_core.py`):** Acts
as the parent caller for the strategies.
*   **`Risk_Manager.py` (Refactored `Risk_Engine.py` & `Chop_Mode.py`):**
Owns gap risk validations, chop detection, and theta dodges.
*   **`Chain_Analyzer.py`:** Dedicated strictly to PCR/Volume crunching.
```

Domain 4: `Presentation & Output Layer` (UI, Terminal, Excel)

```
**Goal:** The outer boundary. Takes data from the engines and formats it
for output.
*   **`Excel_Writer.py` (NEW - split from scanner_excel.py):** Focuses
solely on Excel `.xlsx` / `xlwings` manipulations.
*   **`Terminal_UI.py` (NEW - split from Market_Scanner / MainEngine):**
Owns `print_watchlist_scan()`, `print_heartbeat()`,
`log_rsi_trend_mismatch()`.
*   **`Alerts_Manager.py` (NEW):** Handles Telegram (`Communication.py`)
+ Audio Alerts (`play_alert_sound` from scanner).
```

Domain 5: `The Orchestrator` (The new MainEngine/Scanner)

```
*   **`Market_Orchestrator.py` (Replaces `Market_Scanner`):** Highly
readable. It creates the data loops. It fetches data (from Domain 1),
passes it to Strategy (Domain 3), and passes the result to the Output
Layer (Domain 4). It should contain almost NO heavy logic itself.
*   **`Execution_Engine.py` (Replaces `MainEngine.py` execution loops):**
Only listens for "Go" signals and deals with Order tracking/execution.
```

4. Execution Step 1: Breaking down `scanner\_excel.py` & `Market\_Scanner.py`

To safely reduce these two 800+ line files, we will extract the pure utility functions first (that have no internal dependencies):

1. ****Extract String/Symbol Utilities:**** Move `\_parse\_fno\_symbol()`, `\_normalize\_exchange()`, `\_is\_fno\_symbol()` to a new `symbol\_utils.py`.
2. ****Extract Excel-Only Logic:**** Move `ensure\_sheet\_quote\_headers()`, `\_sheet\_last\_data\_row()`, `refresh\_strike\_window\_on\_websocket()` to a dedicated `excel\_manager.py`.
3. ****Extract Sound Alerts:**** Move `play\_alert\_sound()` and `print\_alert\_sound\_help()` to `Alerts\_Manager.py`.

4. ****Extract Technical Indicators:**** Clean up ``Indicators.py`` and enforce that all modules import RSI/VWAP purely from there.

By separating the "Calculation", the "API Fetch", and the "Console Formatting", the file sizes for ``Market_Scanner.py`` and ``MainEngine.py`` will drop by roughly 60%.